HookCalculator.jsx

```jsx
import React, { useState, useEffect, useCallback, useRef, useMemo, useContext, createContext } from 'react';
import './Calculator.css'; // Import the pure CSS file


const ThemeContext = createContext();
// Context to manage theme state across the calculator
export const CalculatorProvider = ({ children }) => {
  const [isHackerMode, setIsHackerMode] = useState(false);
  const toggleTheme = () => setIsHackerMode(prev => !prev);

  // Provide the theme state and toggle function to the context
  return (
    <ThemeContext.Provider value={{ isHackerMode, toggleTheme }}>
      <div className="app-wrapper">
        {children}
      </div>
    </ThemeContext.Provider>
  );
};

// Main calculator component using React hooks
export const HookCalculator = () => {
  const { isHackerMode, toggleTheme } = useContext(ThemeContext); // Destructure theme state and toggle function

  // State variables for current value, previous value, and operator
  const [currentVal, setCurrentVal] = useState('0');
  const [previousVal, setPreviousVal] = useState(null);
  const [operator, setOperator] = useState(null);

  // Ref to store the history of operations without causing re-renders
  const historyLog = useRef([]);
```

```
// Format the current value for display with commas for thousands
const displayValue = useMemo(() => {
  if (currentVal === 'Error') return currentVal;
  return currentVal.toString().replace(/\B(?=(\d{3})+(?!\d))/g, ","); // Add commas for thousands
}, [currentVal]);

// Handle input for numbers, operators, and special keys
const handleInput = useCallback((val) => {
  if (!isNaN(val) || val === '.') {
    setCurrentVal(prev => (prev === '0' && val !== '.' ? val : prev + val));
  } else if (['+', '-', '*', '/'].includes(val)) {
    setPreviousVal(currentVal);
    setOperator(val);
    setCurrentVal('0');
  } else if (val === '=' || val === 'Enter') {
    if (!operator || !previousVal) return;
    let result = 0;
    const prev = parseFloat(previousVal);
    const current = parseFloat(currentVal);

    if (operator === '+') result = prev + current;
    if (operator === '-') result = prev - current;
    if (operator === '*') result = prev * current;
    if (operator === '/') result = current === 0 ? 'Error' : prev / current;

    const resultStr = String(result);
    historyLog.current.push(`${prev} ${operator} ${current} = ${resultStr}`);

    setCurrentVal(resultStr);
    setPreviousVal(null);
    setOperator(null);
```

```
    } else if (val === 'C' || val === 'Escape') {
      setCurrentVal('0');
      setPreviousVal(null);
      setOperator(null);
    }
  }, [currentVal, previousVal, operator]);

  // Add keyboard event listener for calculator input
  useEffect(() => {
    const handleKeyDown = (e) => {
      const key = e.key;
      if ((key >= '0' && key <= '9') || ['+', '-', '*', '/', '.', 'Enter', 'Escape'].includes(key)) {
        e.preventDefault();
        handleInput(key === 'Escape' ? 'C' : key);
      }
    };
    window.addEventListener('keydown', handleKeyDown); // Add event listener for keydown events
    return () => window.removeEventListener('keydown', handleKeyDown); // Cleanup event listener on component
unmount
  }, [handleInput]);

  // Render the calculator UI with dynamic theme and display
  return (
    <div className={`calc-container ${isHackerMode ? 'theme-hacker' : 'theme-standard'}`}>

      <div className="calc-header">
        <h2 className="calc-title">React Calc_OS</h2>
        <button onClick={toggleTheme} className="theme-btn">
          {isHackerMode ? 'GUI Mode' : 'Terminal Mode'}
        </button>
      </div>
```

```
      <div className="calc-screen">
        <div className="screen-history">
          {previousVal} {operator}
        </div>
        <div className="screen-main">
          {displayValue}
        </div>
      </div>

      <div className="calc-grid">
        {['7', '8', '9', '/'].map(btn => (
          <button key={btn} onClick={() => handleInput(btn)} className="btn btn-standard">{btn}</button>
        ))}
        {['4', '5', '6', '*'].map(btn => (
          <button key={btn} onClick={() => handleInput(btn)} className="btn btn-standard">{btn}</button>
        ))}
        {['1', '2', '3', '-'].map(btn => (
          <button key={btn} onClick={() => handleInput(btn)} className="btn btn-standard">{btn}</button>
        ))}
        {['C', '0', '.', '+'].map(btn => (
          <button key={btn} onClick={() => handleInput(btn)} className={`btn ${btn === 'C' ? 'btn-clear' : 'btn-
standard'}`}>{btn}</button>
        ))}
        <button onClick={() => handleInput('=')} className="btn btn-equal">=</button>
      </div>

      <div className="calc-footer">
        <p>Session History: {historyLog.current.length} operations stored in memory (useRef).</p>
      </div>
    </div>
  );
};
```

App.js

```jsx
import React from 'react';
// Import both components from the single file you created
import { CalculatorProvider, HookCalculator } from './HookCalculator';

export default function App() {
  return (
    <div className="min-h-screen bg-slate-950 flex items-center justify-center p-4">
      {/*
        Wrap the calculator in the Provider.
        This makes the ThemeContext available to the calculator.
      */}
      <CalculatorProvider>
        <HookCalculator />
      </CalculatorProvider>
    </div>
  );
}
```

Calculator.css

```css
/* Container Background */
.app-wrapper {
  min-height: 100vh;
  background-color: #020617;
  display: flex;
  align-items: center;
  justify-content: center;
```

```
  padding: 1rem;
  font-family: monospace;
}

/* Main Calculator Card */
.calc-container {
  padding: 1.5rem;
  border-radius: 0.75rem;
  border: 1px solid;
  box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5);
  max-width: 24rem;
  width: 100%;
  transition: all 0.3s ease;
}

/* Themes */
.theme-standard {
  background-color: #1e293b;
  color: #f1f5f9;
  border-color: #475569;
}
.theme-hacker {
  background-color: #000000;
  color: #22c55e;
  border-color: #15803d;
}

/* Header */
.calc-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
```

```
  margin-bottom: 1rem;
}
.calc-title {
  font-size: 0.875rem;
  font-weight: bold;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  margin: 0;
}
.theme-btn {
  font-size: 0.75rem;
  padding: 0.25rem 0.5rem;
  background: transparent;
  border: 1px solid currentColor;
  color: currentColor;
  border-radius: 0.25rem;
  cursor: pointer;
  opacity: 0.7;
}
.theme-btn:hover { opacity: 1; }

/* Screen */
.calc-screen {
  background-color: #020617;
  padding: 1rem;
  border-radius: 0.25rem;
  margin-bottom: 1rem;
  text-align: right;
  overflow: hidden;
  box-shadow: inset 0 2px 4px 0 rgba(0, 0, 0, 0.5);
}
.screen-history {
```

```
  font-size: 0.75rem;
  color: #64748b;
  height: 1rem;
  margin-bottom: 0.25rem;
}
.screen-main {
  font-size: 1.875rem;
  font-weight: bold;
  letter-spacing: 0.05em;
}

/* Keypad Grid */
.calc-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 0.5rem;
}

/* Buttons */
.btn {
  padding: 0.75rem;
  font-size: 1.25rem;
  border: none;
  border-radius: 0.25rem;
  cursor: pointer;
  color: white;
  font-family: inherit;
  transition: background-color 0.2s;
}
.btn-standard { background-color: #334155; }
.btn-standard:hover { background-color: #475569; }
```

```css
.btn-clear { background-color: #7f1d1d; }
.btn-clear:hover { background-color: #991b1b; }

.btn-equal {
  background-color: #0369a1;
  grid-column: span 4;
  font-weight: bold;
  margin-top: 0.5rem;
}
.btn-equal:hover { background-color: #0284c7; }

/* Footer */
.calc-footer {
  margin-top: 1.5rem;
  padding-top: 1rem;
  border-top: 1px solid #334155;
  font-size: 0.75rem;
  opacity: 0.7;
}
```